# Change Point Detection

University of Bucharest

Kononov Daniil

February 2026

# Abstract

This project is about change point detection in time series and its application in trading. Here I analyze different offline algorithms for change-point detection, describe how to adapt them to online scenarios and, finally, apply them by building a trading trend following system which executes market orders when change-points are detected.

# Contents


**1 Introduction** **3**

**2 Cost Functions** **4**

**3 Cost Computers** **5**

**4 Algorithms** **7**

4.1 General Discussion . . . . . 7
4.2 Binary Segmentation . . . . . 7
4.3 Optimal Partitioning . . . . . 8
4.4 Choosing Penalties . . . . . 8
4.5 Online Wrapper . . . . . 9

**5 Comparison of Algorithms Results** **10**

**6 Linear Trend Trading Strategy** **13**

**7 Conclusion** **15**

# Chapter 1

# Introduction

Change-point detection is a commonly encountered challenge in many different industries such as finance, DevOps, healthcare, etc. The problem consists in identifying certain points in time when the time series behavior significantly changes. In order to detect a change, we need to be able to quantify it numerically. For this purpose, several metrics have been developed to detect changes in mean, variance, slope, etc. In change-point detection, there are two different scenarios: offline and online. Offline detection is easier since you already have access to the entire data and for each potential change-point we know both past and future values of time series. In an online setting change-points should be detected and reported in real-time, which means that we know future values only within a small time frame. The size of this time frame will represent a delay at which change-points will be detected. A shorter delay implies faster detection, which is critical in many situations. However, there is usually a tradeoff between the delay and accuracy of detected change-points. The shorter the delay, the fewer future points observed, and the lower the detection accuracy. That is why different algorithms are being used for online detection depending on the situation. Some offline algorithms can also be adapted to an online setting. [6]

In this paper, I implement and analyze two change-point detection algorithms: Binary Segmentation and Optimal Partitioning. [7] I tested them on two types of offline data: artificial and real time series. Artificial data is generated to have clear changes in behavior. Real data represents financial data related to the stock market, such as prices and returns, over a certain period of time. After testing different change types (changes in mean, variance, mean and variance, slope) and obtaining decent results, I moved on and created an online wrapper for these offline algorithms. Finally, I applied the resulting online detectors by building a trading system which executes buy and sell orders based on slope change detection. By adjusting different parameters such as minimum distance between detected change-points, I managed to create a strategy with Sharpe ratio [9], which slightly beats the performance of simple Buy and Hold strategy on SP500 throughout different periods of time.

# Chapter 2

# Cost Functions

Detecting change-points in time series involves splitting it into a number of segments so that within each segment values possess identical properties. Each segment gets assigned a cost that measures how much its values differ. This cost is being calculated by the cost function which is chosen depending on specific types of change we are interested in. Some cost functions are parametric and are based on maximum likelihood estimation while others are non-parametric (like rank function). [6]

**Mathematical definitions and formulae**: suppose that we are given a time series and need to find a set of K change-points $T^* = (t_1^*, ..., t_K^*)$. We need to minimize the total cost of all K+1 segments:
$(\hat{t}_1, \ldots, \hat{t}_K) = \arg\min_{(t_1,\ldots,t_K)} \sum_{k=0}^{K} c\big(x_{t_k:t_{k+1}}\big)$. [6]
Here $t_0 = 0$ and $t_K = N$ where N is the total number of samples. The cost function measures the homogeneity of the segment. Parametric cost function is defined as:

$$\mathrm{cML}(x_{a:b}) = -\sup_{\theta} \sum_{n=a+1}^{b} \log f(x_n \mid \theta)$$

[6] Below are several cost functions for different change types:

- Change in mean L2 norm cost
  $c_{L_2}(x_{a:b}) = \sum_{n=a+1}^{b} \|x[n] - \mu_{a:b}\|_2^2$ [6]
- Change in mean and variance
  $c_{\Sigma}(x_{a:b}) = (b-a)\log\sigma_{a:b}^2 + \frac{1}{\sigma_{a:b}^2}\sum_{n=a+1}^{b} \|x[n] - \mu_{a:b}\|_2^2$ [6]
- Change in slope and intercept (linear regression)
  $c_{\mathrm{linear}}(x_{a:b}) = \min_{\alpha} \sum_{n=a+1}^{b} \left\| x[n] - \sum_{i=1}^{M} \alpha_i \beta_i[n] \right\|_2^2$
  where $\beta_1[n] = 1$ and $\beta_2[n] = n$ [6]

# Chapter 3

# Cost Computers

In order to be able to quickly compute the cost of a segment in $O(1)$, I introduced CostComputer and LinearCostComputer classes which are later used by algorithms. CostComputer class receives time series and cost type as input where cost type determines the type of changes to be detected by algorithm. Since these costs involve operations with time series properties such as mean and variance, I precomputed prefix sums and prefix squared sums for my time series. Using them I was able to calculate mean and variance on any segment range in constant time using basic arithmetic operations. However, since computation of linear cost for slope change detection involved more complex arithmetic computations, I created a separate LinearCostComputer class. Its cost is a residual sum of squares. Let $y_i = \alpha + \beta x_i + \varepsilon_i$.. Then
$\widehat{\alpha} = \bar{y} - \widehat{\beta}\,\bar{x}$,

$$\widehat{\beta} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2} = \frac{\sum_{i=1}^{n}\Delta x_i \Delta y_i}{\sum_{i=1}^{n}\Delta x_i^2}.$$

Let's compute values

$$S_{xx} = \sum_{i=1}^{n}(x_i - \bar{x})^2$$

,

$$S_{yy} = \sum_{i=1}^{n}(y_i - \bar{y})^2$$

,

$$S_{xy} = \sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})$$

. [10] [4] [3] [8] They can be computed efficiently in $O(1)$ by storing prefix sums for sums, squared sums and product sums for both x and y. In linear regression $TSS = RSS + SS_{reg}$ , so that $RSS = TSS - SS_{reg}$. $TSS = S_{yy}$ , so we need to derive $SS_{reg}$ from the equation. $SS_{reg} = \sum_{i=1}^{n}(\hat{y}_i - \bar{y})^2$. By substituting $\hat{y}_i = \hat{\alpha} + \hat{\beta}x_i$ into the equation it can be shown that $SS_{reg} =$

$\hat{\beta}^2 \sum_{i=1}^{n}(x_i - \bar{x})^2 = \hat{\beta}^2 S_{xx}$. So $SS_{reg} = \left(\frac{S_{xy}}{S_{xx}}\right)^2 S_{xx} = \frac{S_{xy}^2}{S_{xx}^2} S_{xx} = \frac{S_{xy}^2}{S_{xx}}$ which means that $RSS = S_{yy} - \frac{S_{xy}^2}{S_{xx}}$ . These calculations are implemented in cost() function of LinearCostComputer class.

# Chapter 4

# Algorithms

## 4.1 General Discussion

There are several algorithms for change-point detection. Offline detection algorithms usually either accept a fixed number of change-points to be detected as an input parameter or use penalties to consider only significant changes. Some of the methods are CUSUM [5], Binary Segmentation [7], Optimal Partitioning [7], PELT [7], Bayesian Change-point Detection [2] [1]. I implemented two of them: Binary Segmentation and Optimal Partitioning.

## 4.2 Binary Segmentation

Binary Segmentation is a greedy algorithm which recursively searches the best split point for a segment in order to minimize the cost of newly obtained left and right segments. If the number of change-points K is known then the algorithm can greedily select K points which minimize costs of split segments. In situations where the optimal number of change-points K is unknown, a penalized version of binary segmentation can be used. [7]

The main part of the algorithm is to calculate the gain obtained by splitting a segment into two sub-segments. If $c(x[start : end])$ represents the cost of current segment and a split point start¡point¡end is chosen, then the split is worth it only if $c(x[start : point]) + c(x[point + 1 : end]) + \beta < c(x[start : end])$. Here $\beta$ represents the penalty for adding an extra change-point. This equation can be rewritten as $c(x[start : end]) - c(x[start : point]) - c(x[point + 1 : end]) > \beta$ where the left-hand side represents current gain from splitting. If gain is less than penalty then the split does not make sense. I implemented binary segmentation with an extra parameter which ensured a minimum distance between detected change-points. [7]
Below are the steps of the algorithm:

1. Get time series as input and initialize a cost computer.

2. Set a fixed penalty based on input time series.
3. Initialize a list of tuples which represents segments with value (0, N) where N is the number of samples in time series.
4. While current gain is greater than penalty do
5. Iterate over all segments (tuples) in segments list and for each segment calculate the best split which maximizes the gain
6. Pick the best gain out of all gains and if it is higher than penalty, make a new split and update segments list. Also add new change-point to the output list.
7. Return the sorted list of all found change-points.

The time complexity of this implementation with optimized $O(1)$ cost calculation is $O(n^2)$ in the worst case and $O(n \log n)$ in the best case.

## 4.3 Optimal Partitioning

The main disadvantage of Binary Segmentation is that greedy selection of best splits does not always lead to optimal results. The split made on previous iteration affects all the splits made in future iterations. In order to obtain better splits, Optimal Partitioning algorithm can be used instead. It is based on dynamic programming and ensures globally optimal solution. Let N be the number of samples in time series and $\beta$ a fixed penalty. The idea is that we initialize a cost vector C of size N+1 with $C[0] = -\beta$. Then we iterate from 1 to N and for each time t we calculate current optimal partitioning up to this time as $C[t] = min_{0<p<t}[C[p] + c(x[p:t]) + \beta]$ In order to keep track of detected change-points, a path array is introduced which stores the previous optimal split point for current time t. This algorithm works in $O(n^2)$ time complexity, however using PELT pruning it can be improved to work in $O(n)$. [7]

## 4.4 Choosing Penalties

In order to use penalized versions of the above algorithms, some fixed penalty should be established. There are different methods to choose a penalty. Two popular criteria are Bayesian information criterion (BIC) with $BIC = k \log n - 2 \log \hat{L}$ and Akaike Information criterion (AIC) with $AIC = 2k - 2 \log \hat{L}$
where k is the number of model parameters, n is the total number of samples and $\hat{L}$ is maximum likelihood estimation. The penalties I used in this project for different cost types are based on BIC. [6]

## 4.5 Online Wrapper

Previously I described the algorithms I implemented for offline change-point detection. However, since I wanted to apply them also in real-time scenarios, I built an online wrapper for offline algorithms. I created NaiveOnlineDetector class which takes as input algorithm type (Binary Segmentation or Optimal Partitioning), cost type, minimum distance between change-points and horizon size (how much data to keep in buffer, older data might not be relevant). For each new input value of time series it adds it to local buffer and gets the detected change-points by calling an offline algorithm with current buffer values. Since in real-time scenarios I was interested only in the most recent change-point, I grabbed only the last detected change-point. If it was later within minimum distance than the last change-point detected previously, I updated the last change-point to this recent one. Also, if buffer size became bigger that horizon size, I was reducing it to contain only fresh data. In order to keep mapping between global time series indexing and local buffer indexing, I stored in my NaiveOnlineDetector class offset integer field which was keeping track of how many values have been erased from local buffer so far. The class is called naive because this online wrapper implementation is computationally inefficient. However, it was still fast enough to be able to test it on stock market related data.

# Chapter 5

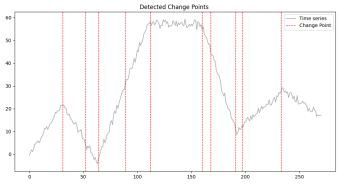
# Comparison of Algorithms Results

Here I test the implemented algorithms both in offline and online scenarios, on artificial and real data. In order to generate artificial data, I created a

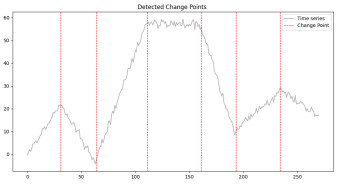
```
generate_series
```

function which generates data with identical properties on the same segment. Depending on the cost type, data within each segment had same statistic like slope, mean, variance. For real data, I used different properties of SP500 ETF like closing prices, log returns, smoothed prices.

In offline scenarios Optimal Partitioning algorithm performed better than Binary Segmentation, being less susceptible to noise. For change in variance both algorithms performed similarly well. However, for change in slope BinSeg algorithm detected some redundant false positives because of the noise. Below are visual results of detected change-points:




(a) BinSeg




(b) Opt

Figure 5.1: Slope change offline detection

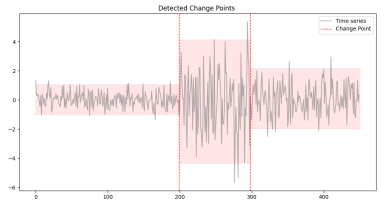



Figure 5.2: Variance changes offline

Testing both algorithms on stock prices offline data with linear cost (trend estimation) does not show significant difference between these 2 algorithms. They both capture major trend changes:

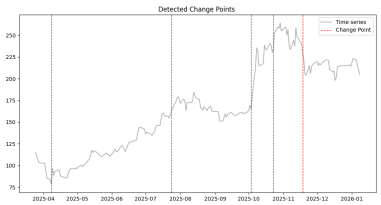
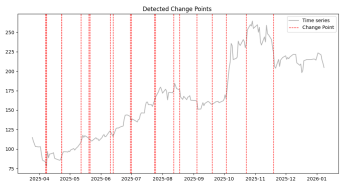
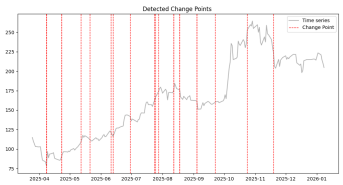



Figure 5.3: Stock prices trend changes offline

However, online detection captures more noise because the algorithms do not have access to the whole picture and need to report changes as soon as they are detected, By increasing minimum distance parameter more noise can be filtered out at the expense of higher detection delay.




(a) BinSeg




(b) Opt

Figure 5.4: Slope change online AMD stock prices with $dist = 10$

By increasing minimum distance from 10 to 15, the number of detections drops significantly, noise gets filtered out:

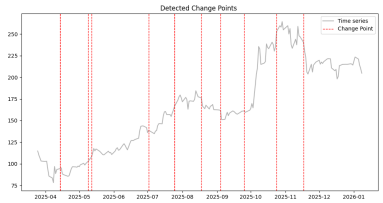



Figure 5.5: Slope change online AMD stock prices with $dist = 15$

As a result, both algorithms are pretty good at detecting change-points. The difference between them is a tradeoff between detection accuracy and computational speed. BinSeg is faster than Opt but more susceptible to noise.

# Chapter 6

# Linear Trend Trading Strategy

Having implemented and tested change-point detection methods, I can now move on and apply them in real world by building a trend following trading strategy. The idea is to trade based on trend change detection. The strategy can be summarized as follows:

1. Initialize online detector with linear cost type, Opt or BinSeg model type, minimum distance and horizon size parameters.
2. For each new closing price data available, update the detector with it and find whether a new change-point has been detected.
3. If new change-point is detected, calculate the angle of the slope of linear regression line fitted to prices in range from detected point to current moment. This angle represents an estimate of trend strength.
4. Update exponential moving average of trend and continue only if current trend is strong enough compared to average.
5. Calculate global trend using a fixed look-back period and continue trading only if local trend change is stronger than global trend.
6. Skip trade if it was a low volume day (compared to average volume from volume look-back window.
7. Set position size proportional to trend to noise ration (calculated as ratio of mean of daily returns to standard deviation of returns).
8. If trend is positive ($angle > 0$) then go long, else go short.

In order to backtest the strategy I used was using SP500 daily data from yahoo finance and backtesting.py library. I tested it on different time periods ranging from 2014 to 2026. As a performance criterion I was using Sharpe ratio

[9]. By adjusting minimum distance parameter of online detector based on the volatility, it can slightly beat a simple buy and hold strategy. For example on data from 2020 to 2026 the average Sharpe ratio for this strategy was 0.87 whereas buy and hold had 0.82 score. In general, the higher the volatility (lower trend to noise ration), the lower minimum distance should be used to quickly react on price changes. However, due to relative simplicity of the strategy and lack of accounting for external events (politics, earnings calls, news, etc.) the average Sharpe ratio for different periods was slightly below 1, although still slightly better than buy and hold.

# Chapter 7

# Conclusion

The goal of the project was to implement and test two change-point detection algorithms and later apply them in a trading strategy. Both algorithms performed decently in different test scenarios in offline and online environment. The choice between BinSeg and Opt algorithms is a tradeoff between high noise tolerance and fast computation. The minimum distance between change-points is a parameter which significantly influenced the amount of detected change-points. It allows to dynamically adjust the output of the algorithm based on problem requirements. Higher minimum distance filters more noise and detects less false positives at the expense of increased delay between the moment change-point appeared and the moment it was reported. After testing the offline performance of these algorithms, an online wrapper has been created for them. Finally, online detector has been applied in a stock trading system which achieved decent results, considering the limitations and relative simplicity of the strategy.

When it comes to algorithms, further improvements can be made. First, Opt algorithm can be optimized by introducing pruning step and transforming it into PELT which has on average $O(n)$ time complexity [7]. Also, NaiveOnlineDetector can, possibly, be optimized to avoid recalculating everything from scratch for every new point, thus reducing computational complexity. Trading strategy itself can also be improved by adding more rules, using indicators and scraping alternative data like news, earnings calls and market sentiment.

# Bibliography


[1] Ryan Prescott Adams and David J. C. MacKay. Bayesian online changepoint detection. *arXiv preprint arXiv:0710.3742*, 2007. URL: https://arxiv.org/abs/0710.3742.

[2] Philip Bretz. Notes on bayesian changepoint detection, 2020. URL: https://math.ou.edu/~pbretz/Changepoint_Detection.pdf.

[3] GeeksforGeeks contributors. Residual sum of squares, 2024. URL: https://www.geeksforgeeks.org/maths/residual-sum-of-squares/.

[4] GeeksforGeeks contributors. Linear regression formula, 2025. URL: https://www.geeksforgeeks.org/maths/linear-regression-formula/.

[5] Pierre Granjon. The cusum algorithm: A short review, 2013. URL: https://hal.science/hal-00914697v1/document.

[6] Laurent Oudre. Lecture 5: Change-point and anomaly detection, 2023. URL: https://www.laurentoudre.fr/ast/AST5.pdf.

[7] Giovanni Romano. Multiple change-point detection, 2025. URL: https://www.lancaster.ac.uk/~romano/teaching/2425MATH337/3_multiple_changes.html.

[8] Eric W. Weisstein. Least squares fitting. URL: https://mathworld.wolfram.com/LeastSquaresFitting.html.

[9] Wikipedia contributors. Sharpe ratio. URL: https://en.wikipedia.org/wiki/Sharpe_ratio.

[10] Wikipedia contributors. Simple linear regression. URL: https://en.wikipedia.org/wiki/Simple_linear_regression.